

Unidad Profesional Interdisciplinaria en Ingeniería y Tecnologías Avanzadas

Alumno

Galicia Cobaxin Daniel

Profesor

Noe Sierra Romero

Unidad de aprendizaje

Aplicaciones distribuidas

Practica 2:

Servicio npm

Fecha de entrega 23/02/2026

Código JS:

```
const express = require('express');
const crypto = require('crypto');
const app = express();
app.use(express.json());

app.use(express.json());
app.use(express.urlencoded({ extended: true }));

const PORT = 3000;

function respuestaOK(data){
  return { success: true, data: data };
}

function respuestaError(msg){
  return { success: false, error: msg };
}

// mascaracteres
app.post('/mascaracteres', (req,res)=>{
  const { cadena1, cadena2 } = req.body;

  if(!cadena1 || !cadena2)
    return res.json(respuestaError("Faltan parámetros"));

  const resultado = (cadena1.length >= cadena2.length)
    ? cadena1 : cadena2;

  res.json(respuestaOK(resultado));
});

// menoscaracteres
app.post('/menoscaracteres', (req,res)=>{
  const { cadena1, cadena2 } = req.body;

  if(!cadena1 || !cadena2)
    return res.json(respuestaError("Faltan parámetros"));

  const resultado = (cadena1.length <= cadena2.length)
    ? cadena1 : cadena2;

  res.json(respuestaOK(resultado));
});

// numcaracteres
app.post('/numcaracteres', (req,res)=>{
  const { cadena } = req.body;

  if(!cadena)
    return res.json(respuestaError("Falta la cadena"));

  res.json(respuestaOK(cadena.length));
});
```

```

// palindroma
app.post('/palindroma', (req,res)=>{
  const { cadena } = req.body;

  if(!cadena)
    return res.json(respuestaError("Falta la cadena"));

  const limpia = cadena.replace(/\s/g,"").toLowerCase();
  const invertida = limpia.split("").reverse().join("");

  res.json(respuestaOK(limpia === invertida));
});
// concat
app.post('/concat', (req,res)=>{
  const { cadena1, cadena2 } = req.body;

  if(!cadena1 || !cadena2)
    return res.json(respuestaError("Faltan parámetros"));

  res.json(respuestaOK(cadena1 + cadena2));
});
// applysha256
app.post('/applysha256', (req,res)=>{
  const { cadena } = req.body;

  if(!cadena)
    return res.json(respuestaError("Falta la cadena"));

  const hash = crypto.createHash('sha256')
    .update(cadena)
    .digest('hex');

  res.json(respuestaOK({
    original: cadena,
    encriptada: hash
  }));
});
// verifysha256
app.post('/verifysha256', (req,res)=>{
  const { cadenaNormal, cadenaEncriptada } = req.body;

  if(!cadenaNormal || !cadenaEncriptada)
    return res.json(respuestaError("Faltan parámetros"));

  const hash = crypto.createHash('sha256')
    .update(cadenaNormal)
    .digest('hex');

  res.json(respuestaOK(hash === cadenaEncriptada));
});
app.listen(PORT, ()=>{
  console.log(`Servidor corriendo en http://localhost:${PORT}`);
});

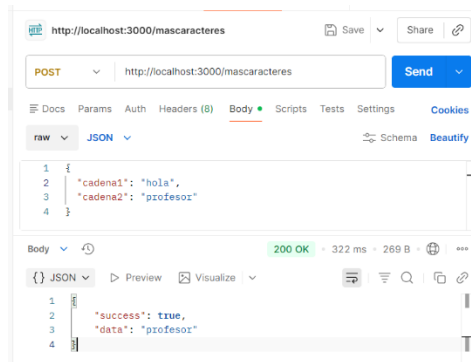
```

Servidor corriendo:

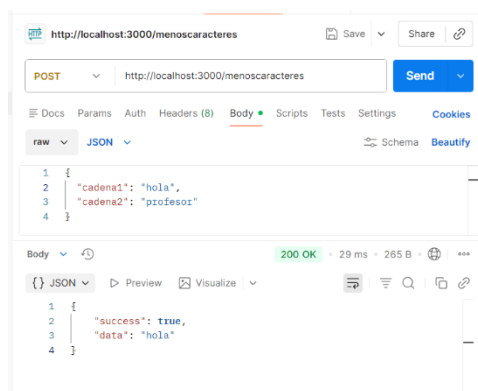
```
PS C:\Users\dgali\servicios-web> node .\server.js
Servidor corriendo en http://localhost:3000
```

Resultados en Postman:

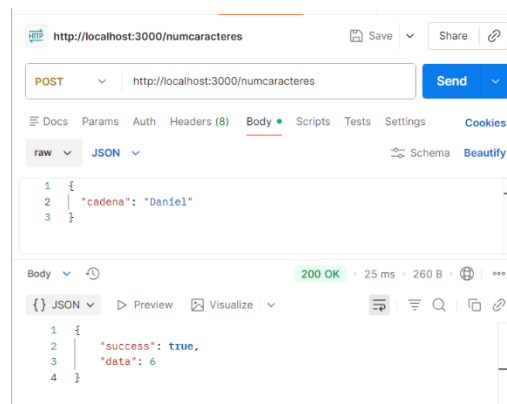
- Servicio “mascaracteres”:



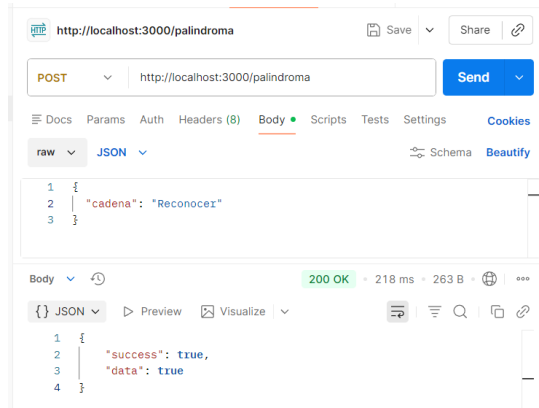
- Servicio “menoscaracteres”:



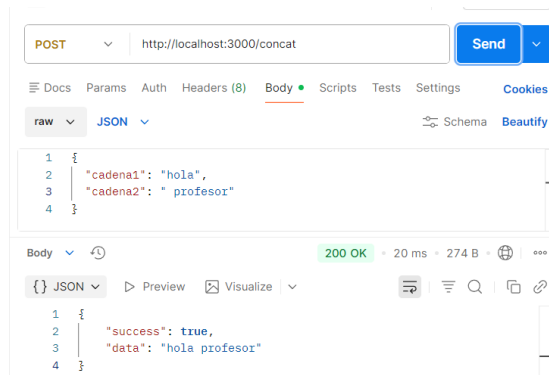
- Servicio “numcaracteres”:



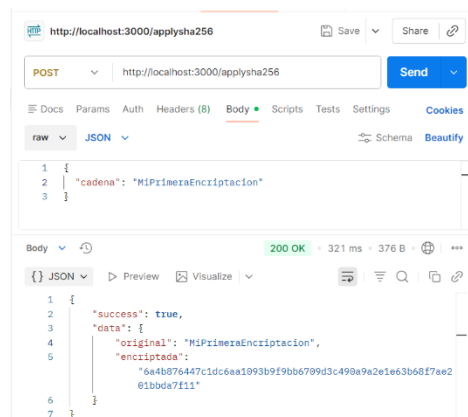
- Servicio “palindroma”:



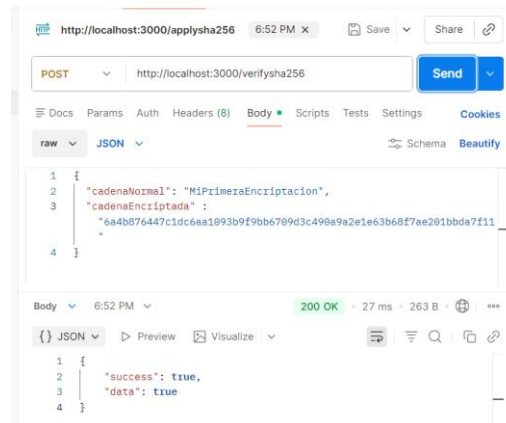
- Servicio “concat”:



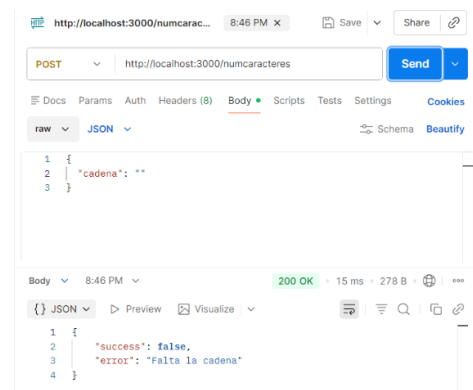
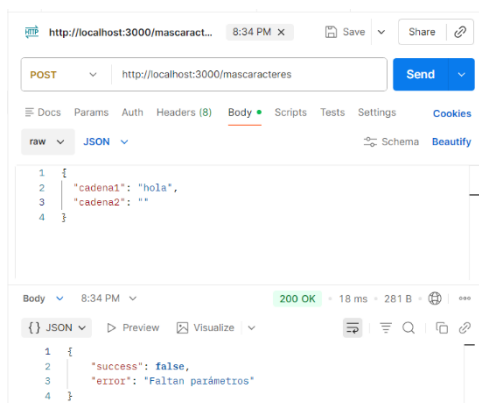
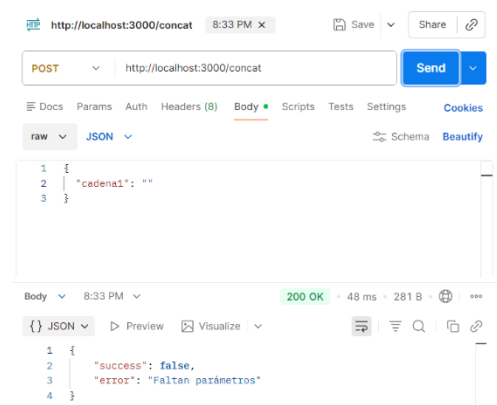
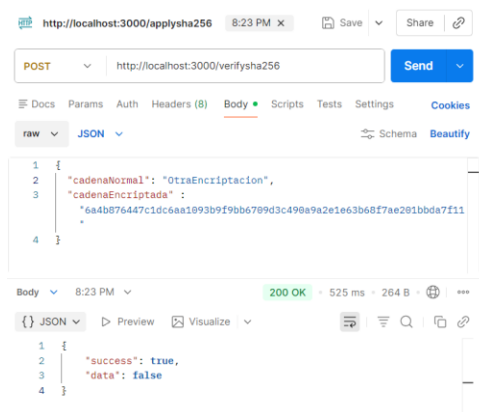
- Servicio “applysha256”:



- Servicio “verifysha256”:



Some exception cases:



Conclusiones

En esta práctica aprendí a utilizar Postman para probar rutas de un servidor hecho con Node.js y Express. Al principio tuve errores como el 500, pero entendí que muchas veces el problema no es la herramienta sino cómo se están enviando los datos o cómo el servidor los está recibiendo.

También comprendí la importancia de que los nombres de las variables que se envían en el body coincidan exactamente con los que el backend espera, ya que, si no coinciden, el servidor puede marcar que falta información, aunque sí se haya enviado algo.

Otra cosa importante fue entender cómo funciona `req.body` y por qué es necesario usar `express.json()` para que el servidor pueda leer datos en formato JSON. Sin esa configuración, los datos llegan como `undefined` y el programa falla.

En general, la práctica me ayudó a entender mejor la comunicación entre cliente y servidor, cómo depurar errores y cómo analizar los mensajes que devuelve el sistema para encontrar la causa del problema.